

Universidad Nacional Autónoma de México

Facultad de Ingeniería



Sistemas Distribuidos

Semestre 2027 - 1

Laboratorio S11. Mexi Banco: una verdad, varias copias.

Alumno: Fernández Herrera Mauricio

Equipo: Pumaverso Distribuido.

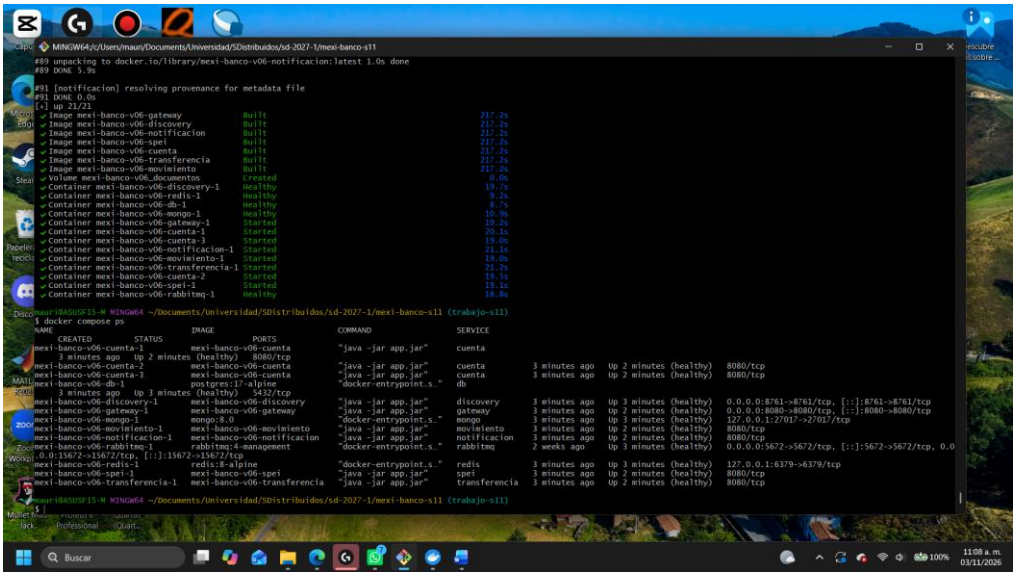
Profesor: ING. OSCAR MANUEL RUIZ HURTADO

Grupo: 02

Entrega: 07/11/2026

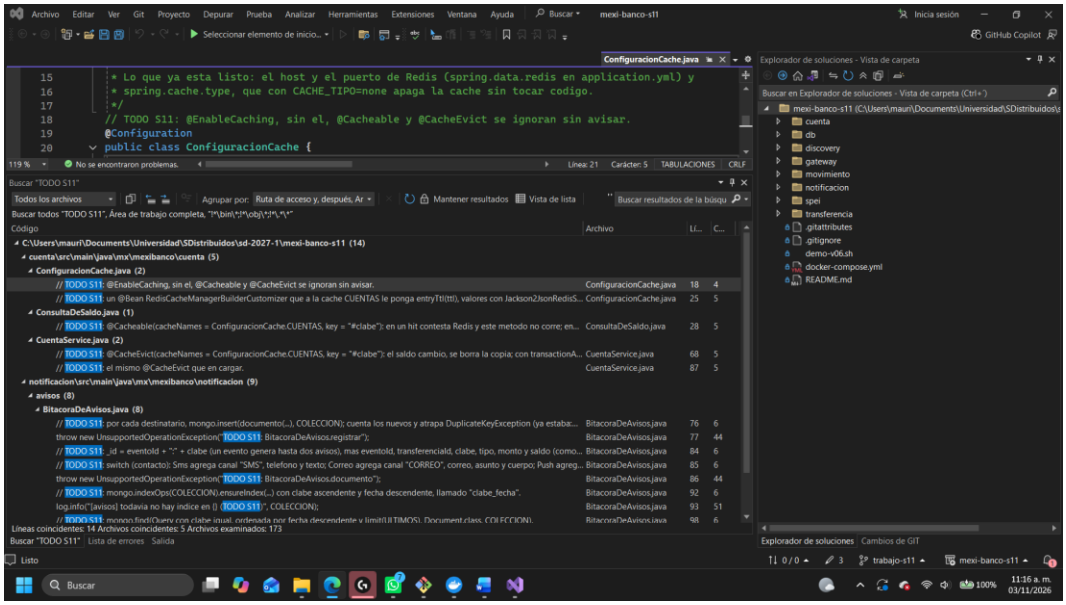
1.- Antes de clase, en tu casa 03/11/2026 – 11:00 am

Descargamos la rama de este lab y corremos todo por primera vez. L terminar, verificamos que todos los servicios digan Healthy.



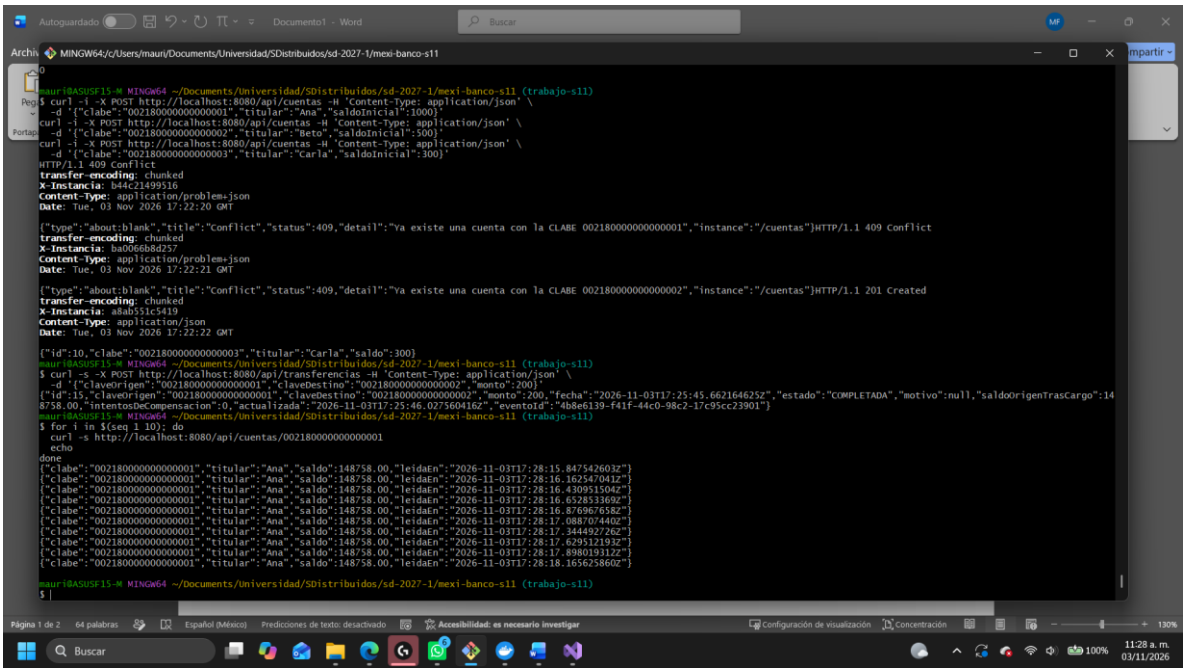
2.- El mapa del código - 03/11/2026 – 11:10 am

Usando VS Code consultamos los archivos con la búsqueda de “TODO S11”.

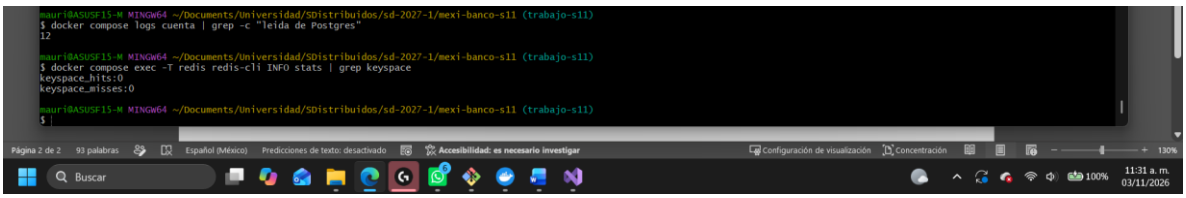


3.- Levantar y ver el punto de partida - 03/11/2026 – 11:20 am

Tras verificar que MongoDB y Redis contestan, creamos las cuentas nuevas, dos de ellas ya existían. Hacemos la transferencia de 200 y consultamos el saldo de Ana 10 veces.

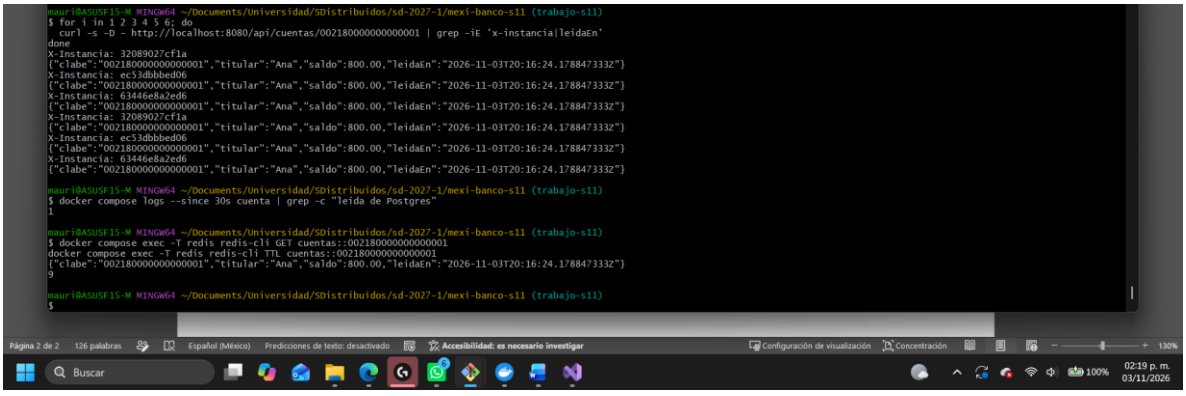


Consultamos la cuenta en Logs y las lecturas erradas y acertadas de Redis.

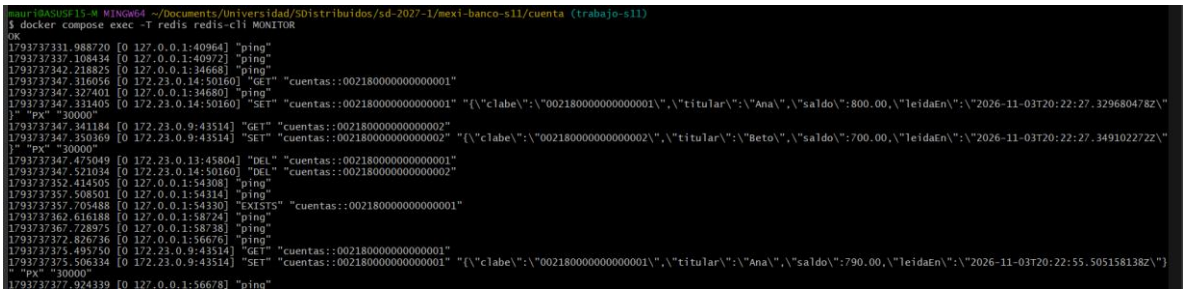


4.- La caché del saldo en Redis - 03/11/2026 – 2:10 pm

Tras realizar todas las modificaciones a los archivos especificadas comprobamos las lecturas de PostgrSQL, el saldo en Redis y el TTL, vemos que los resultados son los esperados.



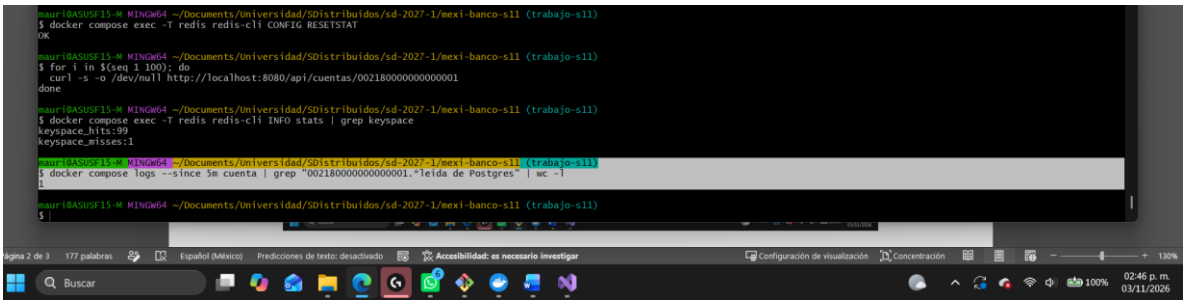
Dejamos corriendo una segunda terminal para wue muestre cada comando que recibe Redis. En la oprimera terminal, transferimos 10 de Ana Beto. Verificamos y vemos que se borran las copias al transferir.



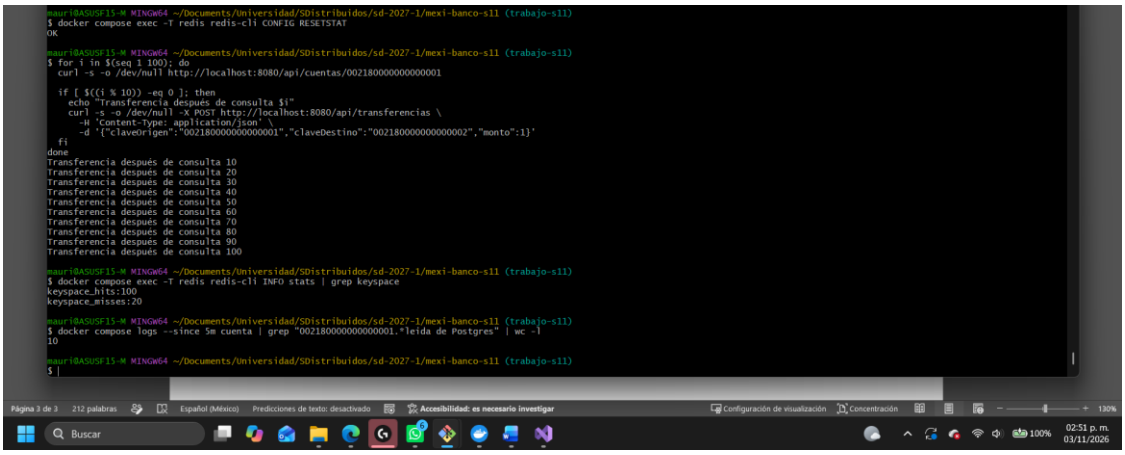


¿Qué compramos con la cache?, para comprobarlo, ejecutamos los siguientes escenarios:

A: Realizamos 100 consultas HTTP de Ana. Al verificar el hit ratio obtenido, vemos que obtuvimos 99 aciertos y 1 fallo en Retis.

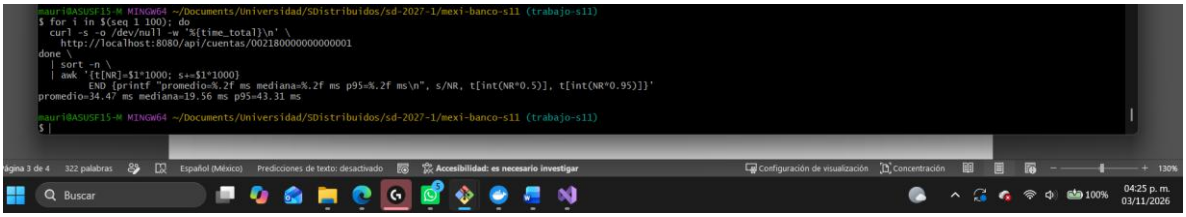


B: Ahora queremos observar qué sucede cuando modificamos los saldos mientras seguimos consultándolos. Realizamos 100 consultas a Ana y, después de cada grupo de diez, transfiere $1 de Ana a Beto.



Notamos que el hit ratio se redujo a un 83.33%. De 100 perdimos 20 hits, además de tener 10 lecturas de Ana en PostgreSQL. La diferencia se explica por la invalidación de caché. cada transferencia elimina las entradas de Ana y Beto. Como el escenario B consulta únicamente a Ana, después de cada transferencia su siguiente consulta debe volver a PostgreSQL. Por eso tenemos 10 fallos adicionales respecto al escenario A.

Seguimos con la medición de la latencia con caché.

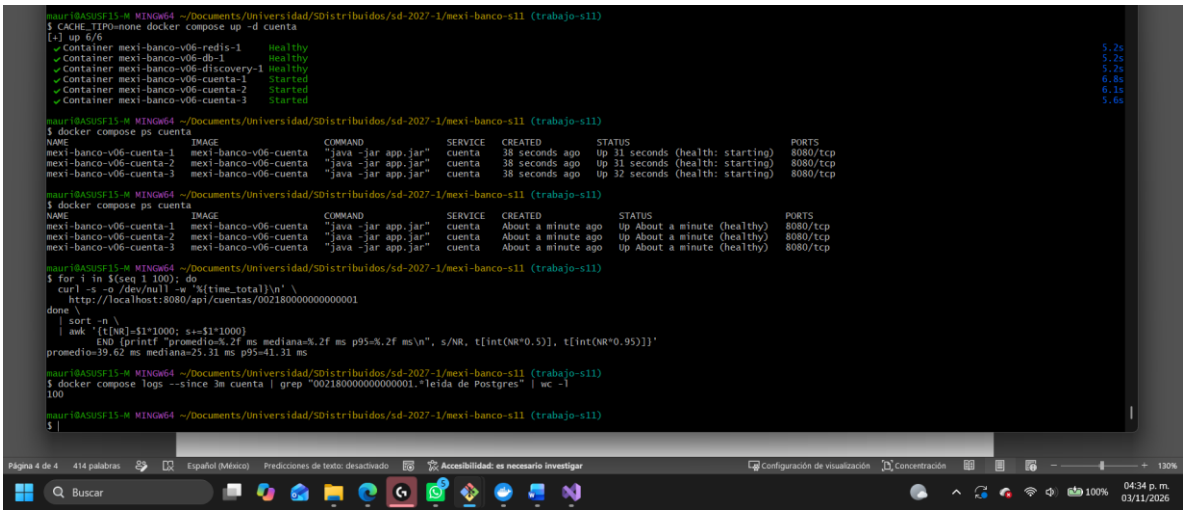


Los resultados fueron:

- Primera medición: promedio de 34.47 ms, mediana de 19.56 ms y p95 de 43.31 ms.
- Segunda medición: promedio de 21.95 ms, mediana de 16.98 ms y p95 de 33.76 ms.
- Tercera medición: promedio de 18.10 ms, mediana de 16.40 ms y p95 de 33.30 ms.

Se observa una disminución en los tiempos de respuesta entre las mediciones, especialmente en el promedio. Esto muestra que Redis permite atender las consultas sin acceder constantemente a PostgreSQL.

Ahora haremos tres mediciones sin cache. Lo apagamos usando CACHE_TIPO=none docker compose up -d cuenta



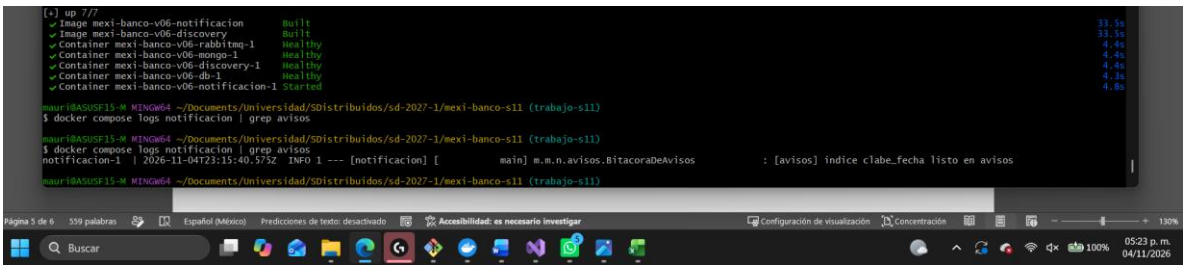
Se realizaron tres mediciones de 100 consultas al saldo de Ana sin utilizar Redis. Los resultados fueron:

- Primera medición: promedio de 39.62 ms, mediana de 25.31 ms y p95 de 41.31 ms.
- Segunda medición: promedio de 25.21 ms, mediana de 22.55 ms y p95 de 39.92 ms.
- Tercera medición: promedio de 26.33 ms, mediana de 24.66 ms y p95 de 39.10 ms.

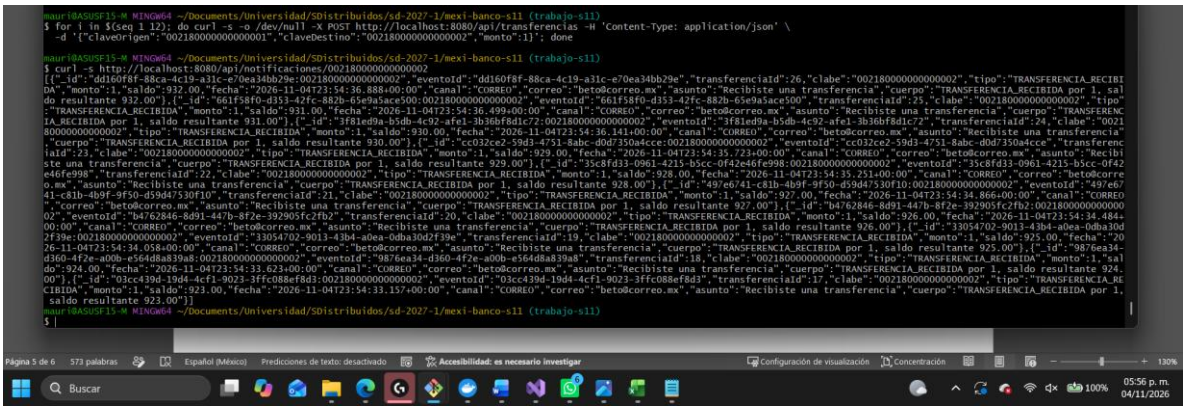
En las tres mediciones, PostgreSQL recibió las 100 consultas. Esto demuestra que, sin caché, todas las solicitudes deben acceder directamente a la base de datos.

5.- Los avisos en MongoDB - 04/11/2026 – 5:01 pm

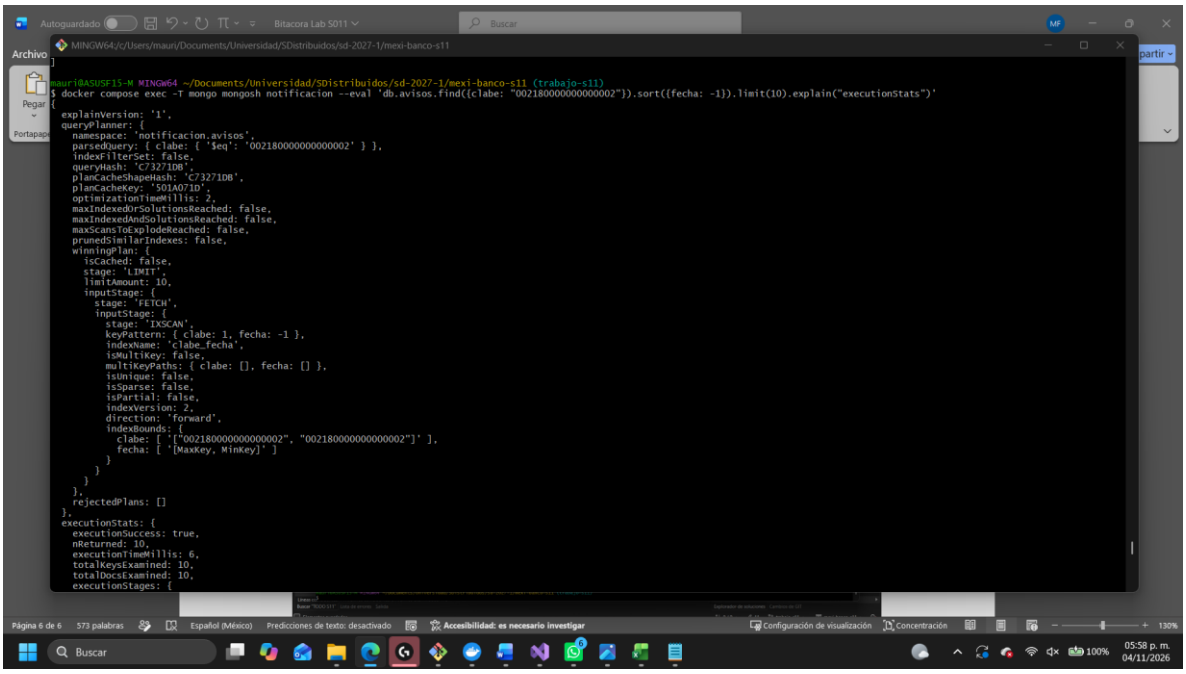
Tras haber armado el aviso con su switch entre diferentes casos, alterar registrar(evento) para guardar sin duplicar, modificar crearIndice() y ultimos(clabe), conectamos el consumidor y volvemos a construir la imagen de notificación.
Buscamos la lína del índice en notificación con el comando docker compose logs notificacion | grep avisos

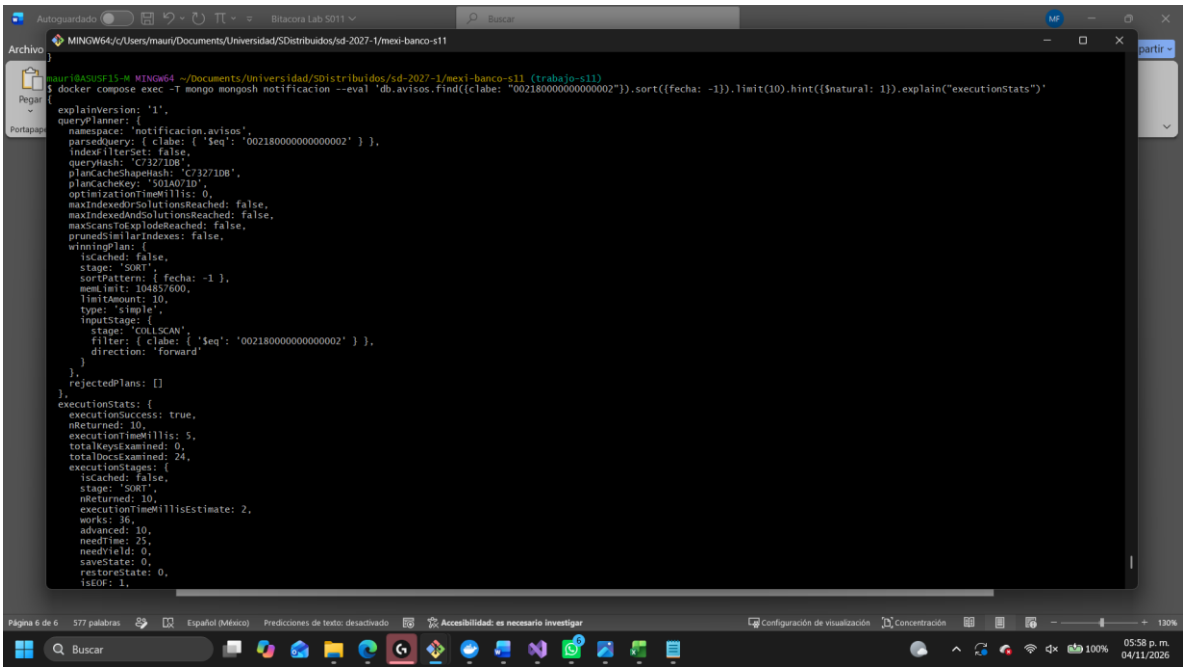


Hacemos 12 transferencias pequeñas de Ana a Beto y pedimos la bitácora de Beto.



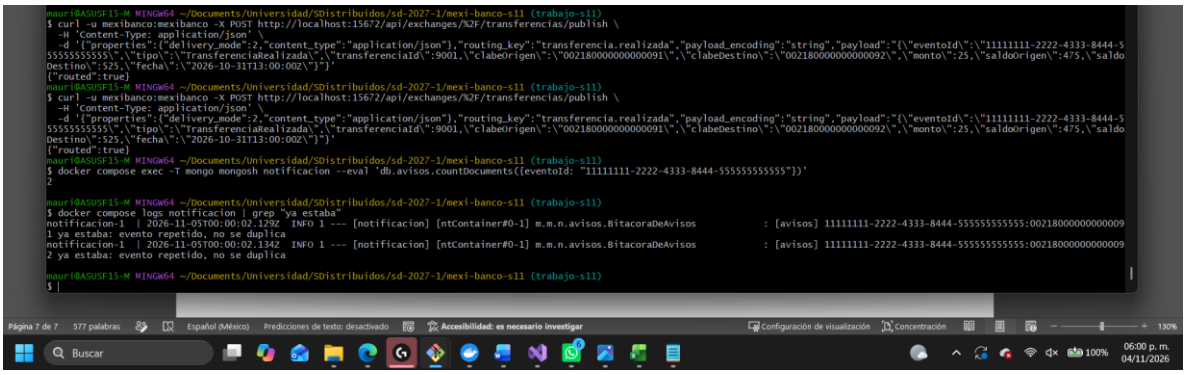
Consultamos los dos explain:





6.- Rompe algo: el evento duplicado - 04/11/2026 – 5:46 pm

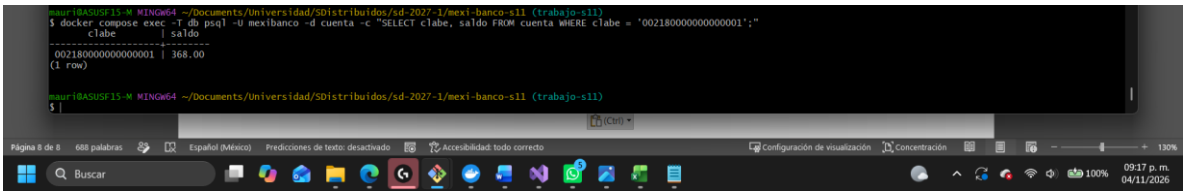
Publicamos dos veces el mismo evento en RabbitMQ utilizando CLABEs de prueba para evitar afectar las cuentas reales. Después, consultamos MongoDB para comprobar cuántos avisos se habían almacenado y revisamos los logs del servicio de notificaciones.



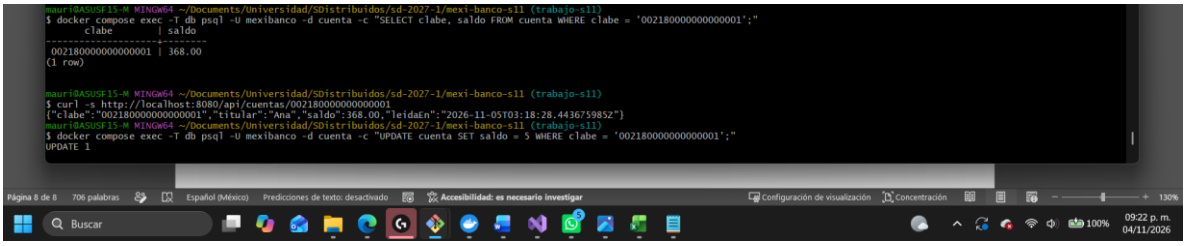
Obtuvimos únicamente dos documentos, uno por cada cuenta, y los mensajes del log confirmaron que el segundo evento fue identificado como repetido y no generó avisos duplicados. Con esto comprobamos que el sistema puede manejar la entrega repetida de mensajes sin duplicar los registros.

7.- Rompe algo: el dato viejo - 04/11/2026 – 9:15 pm

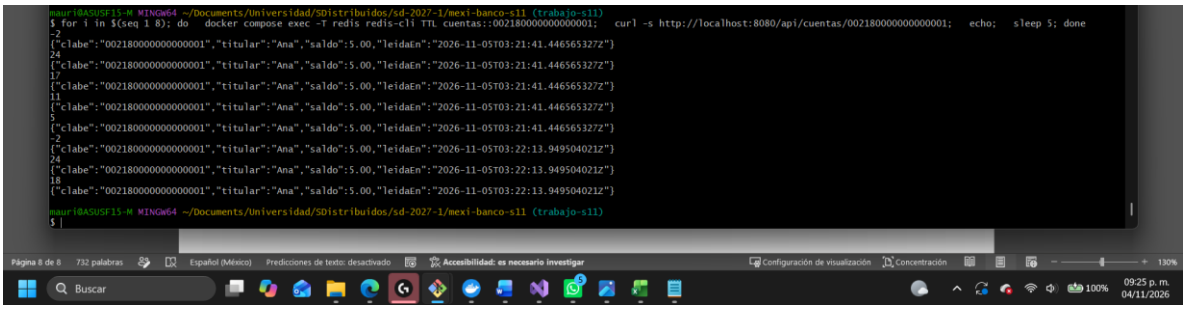
Consultamos el saldo de Ana, el cual es de 368. Lo guardaremos como referencia para restaurarlo exactamente al terminar la prueba.



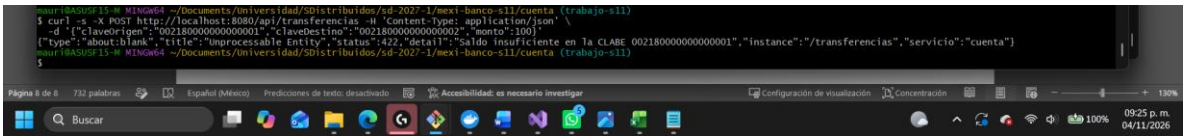
Consultamos el mismo saldo mediante el Gateway para que se guarde en Redis. Modificamos el saldo en PostgrSQL.



Consultamos cada 5 segundos el tiempo de vida restante o TTL de la entrada en Redis.



En otra terminal, ejecutamos la transferencia solicitada. Esta es denegada ya que la autorización del cargo consultó PostgreSQL y encontró únicamente $5.00. Por eso no permitió transferir los $100.



Preguntas de cierre

1. ¿Por qué MongoDB para los avisos y no para el saldo?

MongoDB es adecuado para los avisos porque cada uno puede tener una estructura diferente dependiendo del canal de comunicación, por ejemplo, correo, SMS o notificación de tipo PUSH. Además, normalmente se consultan como un historial de los avisos de cada cuenta. En

cambio, el saldo necesita mantener consistencia y precisión en todo momento, ya que se utiliza para autorizar operaciones financieras. Por eso PostgreSQL es la fuente principal de los saldos.

2. ¿Qué hit ratio mediste en A y en B, y por qué cambia? Con tus números de latencia y de lecturas a Postgres: ¿qué compraste con la caché, velocidad o capacidad?

En el escenario A obtuve un hit ratio del 99%, mientras que en el escenario B fue de 83.33%. La diferencia se debe a que en el segundo escenario se realizaban transferencias constantemente, lo que provocaba que las entradas de Redis se eliminaran y tuvieran que consultarse nuevamente en PostgreSQL. También observé que las lecturas a la base de datos disminuyeron considerablemente con la caché. Aunque las mediciones de latencia mejoraron en general, la diferencia no fue tan grande ni constante. Por lo tanto, considero que principalmente ganamos capacidad al reducir las consultas a PostgreSQL, más que velocidad realmente.

3. ¿Qué pasaría si la autorización del cargo leyera el saldo de la caché? Usa lo que viste en la Parte 4.

Podrían autorizarse transferencias utilizando un saldo desactualizado. En la Parte 4 modificamos directamente el saldo de Ana en PostgreSQL a $5.00, mientras que Redis todavía podía conservar los $368.00 anteriores. Si la autorización hubiera consultado la caché, podría haber permitido una transferencia de $100 que realmente no tenía fondos suficientes para realizarse. Sin embargo, la operación fue rechazada con un error 422 porque la autorización consultó PostgreSQL.

4. Al aplicar un movimiento, ¿por qué se hace DEL de la copia y no SET con el saldo nuevo? ¿Y por qué después del commit y no antes?

Se utiliza DEL porque Redis funciona como una caché y no como la fuente principal de información. Al eliminar la copia, la siguiente consulta obtiene el saldo actualizado directamente de PostgreSQL. Además, se hace después del commit para asegurar que los cambios ya estén guardados. Si se eliminara antes, otra consulta podría recuperar el saldo anterior de PostgreSQL y volver a almacenarlo en Redis, provocando que la caché quedara desactualizada.